

1) Embedded System - Linking

a) Static or Dynamic Linking.

⇒ Dynamic Linking is more suitable because the system has only 64 MB memory. Required modules are loaded only when needed, and shared libraries can be used by multiple applications.

b) Memory and Storage Comparison.

* Static linking: Larger executable and higher memory/storage usage.

* Dynamic linking: Smaller executable and lower memory/storage because libraries can be shared.

c) Disadvantages.

* Dynamic linking has run-time overhead because libraries must be loaded and linked when the program executes.

d) Justification.

* Dynamic linking is preferred because it saves memory and storage and loads only the required modules, which is important for a memory-limited embedded system.

2) CLOUD SERVER - MEMORY ALLOCATION:

a) Configuration Allocation and Paging.

⇒ Contiguous allocation stores each process in one continuous block of physical memory.

- * Simple to implement.

- * Can cause external fragmentation.

- * Difficult to allocate large blocks when memory is scattered.

⇒ Paging divides processes into pages and physical memory into frames.

- * Pages can be placed in different frames.

- * Reduces external fragmentation.

- * Uses physical memory more efficiently.

b) Better Memory Utilization.

⇒ Paging provides better memory utilization because a process does not need a single continuous block of memory.

c) Non-Contiguous memory.

- * Paging divides a process into fixed-size pages and physical memory into frames. The pages can be stored in different frames, and the page table keeps track of their locations.

d) Justification.

⇒ Paging is suitable because it:

- * Reduces external fragmentation.

- * Allows non-contiguous memory allocation.

- * Makes better use of available physical memory.

Application - Paging Vs Segmentation.

a) Suitable Technique

Segmentation:- Segmentation is more suitable because the application consists of logically independent components such as:

- * Code
- * Global data
- * Stack
- * Heap.
- * Shared libraries.

* Each component can be represented as a separate segment.

b) Protection and Controlled Access.

⇒ Each segment can have different access permissions.

For example:

- * Code: Read and Execute
- * Data: Read and Write
- * Stack: Read and Write.

This prevents unauthorized access between different sections.

c) Limitations:-

* The main limitation of segmentation is external fragmentation, because segments have different sizes and may leave unused spaces in memory.

d) Justification.

⇒ Segmentation is preferred because it provides:

- * Logical organization of memory.
- * Different protection levels.
- * Controlled access to code and data.

4) Modern Operating System.

a) Comparison.

Feature	Paging	Segmentation
Memory division	Fixed sized pages	Variable-Size Segments
Protection	Good	Very good
Fragmentation	Internal	External
Logical organization	Low	High
Memory utilization	High	Moderate
Virtual memory	Excellent	Supported

b) Best Mechanisms:

⇒ Segmentation while paging provides the best combination of:

- * Protection
- * Flexibility
- * Efficient memory utilization.
- * Virtual memory support.

c) Logical and Physical memory

* Logical memory is first divided into segments such as code, data and stack. Each segment is then divided into pages.

* These pages are mapped to available physical frames, so the pages need not to be stored continuously.

d) Why others are less suitable.

* Paging: Excellent memory utilization but does not provide strong logical organization.

* Segmentation :- Provides logical organization and protection but suffers from external fragmentation.

* Segmentation with paging : Combines the advantages of both and is therefore more suitable for a modern OS.

5) System A and System B.

a) Suitability of each Technique.

Technique	System A - Gaming PC	System B - Multi user server
Contiguous Allocation	Highly suitable	Less suitable
Paging	Suitable	Highly suitable
Segmentation.	Suitable	Moderately suitable.

b) Most Appropriate Mechanism.

System A - Contiguous Allocation.

- * Large RAM is available.
- * Fast memory access is important.
- * Address translation overhead is low.

System B - paging.

- * Supports many processes.
- * Handles different memory requirements.
- * Reduces external fragmentation.
- * Allows non-contiguous allocation.

c) Comparison.

Factor	Contiguous	Paging	Segmentation.
Fragmentation	External	Internal	External
Memory Utilization	Moderate	High	Moderate
Protection	Low	Good	High
Translation Overhead	Low	Higher	Moderate
Flexibility.	Low	High	High.

d) Final Recommendation.

System A - Contiguous Allocation.

⇒ Recommended because the gaming system has large RAM and requires fast memory access with low address translation overhead.

System B - Paging:

⇒ Recommended because the server handles many processes with varying memory requirements. Paging provides better memory utilization, flexibility and reduced external fragmentation.